

Distributed Systems: Replication and Consistency

CISS 2026WS - Exam-focused course notes

1. Why systems replicate data

Replication places copies of data on multiple nodes to improve availability, fault tolerance, and read scalability. It introduces coordination cost and the possibility that replicas temporarily disagree.

Synchronous replication waits for required replicas before acknowledging a write, giving stronger guarantees at higher latency. Asynchronous replication acknowledges earlier but can lose recent writes if the primary fails.

Review checkpoint: distinguish the guarantees, mechanisms, and failure assumptions introduced on this page.

2. Strong consistency models

Linearizability makes every operation appear to occur atomically between invocation and response, while respecting real-time order. Once a write completes, later reads must see it or a newer value.

Sequential consistency preserves each process's program order and provides one common operation order, but that common order need not respect real-time order.

Linearizability is compositional: independently linearizable objects compose into a linearizable system.

Review checkpoint: distinguish the guarantees, mechanisms, and failure assumptions introduced on this page.

3. Client-centric and eventual consistency

Eventual consistency guarantees that replicas converge if no new updates occur. It does not specify how quickly convergence happens or which intermediate values a read observes.

Read-your-writes ensures a client sees its own completed updates. Monotonic reads prevent a client from observing an older version after observing a newer one.

Session guarantees are often implemented with sticky sessions or version metadata carried between requests.

Review checkpoint: distinguish the guarantees, mechanisms, and failure assumptions introduced on this page.

4. Quorum replication

For N replicas, a read quorum R and write quorum W overlap when $R + W > N$. Two write quorums overlap when $W > N/2$. These inequalities make at least one replica common to the relevant operations.

With $N = 5$, $R = 3$ and $W = 3$ satisfy both overlap conditions. $R = 1$ and $W = 5$ favors cheap reads, while $R = 5$ and $W = 1$ does not guarantee overlapping writes.

Quorum overlap alone does not automatically provide linearizability; version selection, failure handling, and concurrent-write resolution must also be correct.

Review checkpoint: distinguish the guarantees, mechanisms, and failure assumptions introduced on this page.

5. Primary-backup replication

A primary orders writes and forwards them to backups. A failover protocol chooses a new primary when the old one becomes unavailable.

Split brain occurs when multiple nodes believe they are primary and accept conflicting writes. Leases, fencing tokens, and quorum-based leadership prevent stale primaries from continuing to modify protected resources.

A fencing token is monotonically increasing; storage rejects operations carrying a token older than the newest token it has observed.

Review checkpoint: distinguish the guarantees, mechanisms, and failure assumptions introduced on this page.

6. Conflict detection and repair

Version vectors track causal update histories. Two versions are concurrent when neither version vector dominates the other.

Read repair updates stale replicas discovered during reads. Anti-entropy compares replicas in the background, often using Merkle trees to locate differing key ranges efficiently.

Last-write-wins is simple but depends on timestamp assumptions and may discard concurrent updates. Application-specific merge functions can preserve more information.

Review checkpoint: distinguish the guarantees, mechanisms, and failure assumptions introduced on this page.

7. CAP and network partitions

During a network partition, a replicated system cannot guarantee both linearizable consistency and availability for every request. A CP design may reject requests that cannot reach a quorum; an AP design may accept operations that require later conflict resolution.

CAP concerns behavior during partitions, not a permanent three-way design menu. When the network is healthy, systems still make latency, consistency, and durability tradeoffs.

Review checkpoint: distinguish the guarantees, mechanisms, and failure assumptions introduced on this page.

8. Comparing guarantees

Linearizability is appropriate when real-time ordering matters, such as lock ownership or unique allocation. Eventual consistency is suitable when temporary divergence is acceptable and availability or low latency is more important.

Read-your-writes is weaker than linearizability because it protects one client's session, not a single real-time order for all clients.

Exam method: identify the required observable guarantee first, then choose coordination and replication mechanisms capable of enforcing it under the stated failures.

Review checkpoint: distinguish the guarantees, mechanisms, and failure assumptions introduced on this page.